```
#Final Draft
# Meeting Analyzer for Google Colab
# Run this cell first to install dependencies
!pip install openai-whisper transformers moviepy torch -q

import torch
import whisper
import moviepy.editor as mp
from transformers import pipeline
from datetime import timedelta
import json
import re
from collections import Counter
import warnings
import os
from google.colab import files
warnings.filterwarnings('ignore')

device = 0 if torch.cuda.is_available() else -1
print("GPU Available:", torch.cuda.is_available())
print("Device:", "GPU" if torch.cuda.is_available() else "CPU")
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 803.2/803.2 kB 20.7 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for openai-whisper (pyproject.toml) ... done
/usr/local/lib/python3.12/dist-packages/moviepy/config_defaults.py:47: SyntaxWarning: invalid escape sequence '\P'
  IMAGEMAGICK_BINARY = r"C:\Program Files\ImageMagick-6.8.8-Q16\magick.exe"
/usr/local/lib/python3.12/dist-packages/moviepy/video/io/ffmpeg_reader.py:294: SyntaxWarning: invalid escape sequence '\d'
  lines_video = [l for l in lines if ' Video: ' in l and re.search('\d+x\d+', l)]
/usr/local/lib/python3.12/dist-packages/moviepy/video/io/ffmpeg_reader.py:367: SyntaxWarning: invalid escape sequence '\d'
  rotation_lines = [l for l in lines if 'rotate          :' in l and re.search('\d+$', l)]
/usr/local/lib/python3.12/dist-packages/moviepy/video/io/ffmpeg_reader.py:370: SyntaxWarning: invalid escape sequence '\d'
  match = re.search('\d+$', rotation_line)
WARNING:py.warnings:/usr/local/lib/python3.12/dist-packages/moviepy/video/io/sliders.py:61: SyntaxWarning: "is" with 'str' liter
  if event.key is 'enter':

GPU Available: True
Device: GPU
```

```
#Final Draft
class MeetingAnalyzer:
    def __init__(self):
        self.whisper_model = None
        self.summarizer = None
        self.sentiment_analyzer = None
        self.initialize_models()

    def initialize_models(self):
        """Initialize all models once to avoid reloading"""
        print("🔄 Loading AI models...")
        self.whisper_model = whisper.load_model("base", device="cuda" if torch.cuda.is_available() else "cpu")
        self.summarizer = pipeline("summarization", model="facebook/bart-large-cnn", device=device)
        self.sentiment_analyzer = pipeline("sentiment-analysis", device=device)
        print("✅ Models loaded successfully!")

    def transcribe_audio(self, file_path):
        """Convert mp3/mp4 to text using Whisper with timestamps"""
        try:
            # Handle video files by extracting audio
            if file_path.lower().endswith((".mp4", ".avi", ".mov", ".mkv", ".webm")):
                print("🎬 Extracting audio from video...")
                clip = mp.VideoFileClip(file_path)
                audio_path = "temp_audio.wav"
                clip.audio.write_audiofile(audio_path, verbose=False, logger=None)
                clip.close()
                transcribe_path = audio_path
            else:
                transcribe_path = file_path

            print("🎤 Transcribing audio...")
            result = self.whisper_model.transcribe(transcribe_path, word_timestamps=True)

            # Clean up temporary file
```

```
            if transcribe_path == "temp_audio.wav" and os.path.exists("temp_audio.wav"):
                os.remove("temp_audio.wav")

            return {
                "text": result["text"],
                "segments": result.get("segments", []),
                "duration": result.get("segments", [{}])[-1].get("end", 0) if result.get("segments") else 0
            }
        except Exception as e:
            print(f"❌ Error in transcription: {e}")
            return {"text": "", "segments": [], "duration": 0}

    def chunk_text(self, text, max_length=500):
        """Split text into chunks for better processing"""
        sentences = re.split(r'[.!?]+', text)
        chunks = []
        current_chunk = ""

        for sentence in sentences:
            if len(current_chunk + sentence) < max_length:
                current_chunk += sentence + ". "
            else:
                if current_chunk:
                    chunks.append(current_chunk.strip())
                current_chunk = sentence + ". "

        if current_chunk:
            chunks.append(current_chunk.strip())

        return chunks

    def analyze_text(self, transcript_data):
        """Enhanced analysis with better metrics"""
        transcript = transcript_data["text"] if isinstance(transcript_data, dict) else transcript_data
        duration = transcript_data.get("duration", 0) if isinstance(transcript_data, dict) else 0

        if not transcript.strip():
            return self.empty_results()

        print("📊 Analyzing content...")

        # Chunk text for better processing
        chunks = self.chunk_text(transcript)

        # Generate summary
        try:
            if len(transcript) > 100:
                summary_chunks = []
                for chunk in chunks[:3]:  # Limit to first 3 chunks for summary
                    if len(chunk) > 500:
                        summary = self.summarizer(chunk, max_length=500, min_length=30, do_sample=False)
                        summary_chunks.append(summary[0]['summary_text'])

                summary = " ".join(summary_chunks)
            else:
                summary = transcript[:200] + "..." if len(transcript) > 200 else transcript
        except Exception as e:
            print(f"⚠️ Summarization error: {e}")
            summary = transcript[:200] + "..." if len(transcript) > 200 else transcript

        # Sentiment analysis
        sentiment_scores = []
        wasted_reasons = []

        for chunk in chunks:
            if not chunk.strip():
                continue

            try:
                sentiment = self.sentiment_analyzer(chunk[:512])[0]
                sentiment_scores.append(sentiment)

                # Detect potential issues
                if sentiment['label'] == 'NEGATIVE' and sentiment['score'] > 0.8:
                    wasted_reasons.append(f"Highly negative content detected")

            except Exception as e:
```

```
                print(f"⚠️ Sentiment analysis error: {e}")
                continue

        # Calculate metrics
        if sentiment_scores:
            positive_count = sum(1 for s in sentiment_scores if s['label'] == 'POSITIVE')
            negative_count = sum(1 for s in sentiment_scores if s['label'] == 'NEGATIVE')
            neutral_count = len(sentiment_scores) - positive_count - negative_count

            overall_sentiment = self.determine_overall_sentiment(positive_count, negative_count, neutral_count)
            productivity_score = (positive_count + neutral_count) / len(sentiment_scores) * 100
        else:
            overall_sentiment = "Neutral"
            productivity_score = 50
            positive_count = negative_count = neutral_count = 0

        # Time analysis
        actual_duration = duration if duration > 0 else len(transcript.split()) / 150 * 60  # 150 wpm estimate
        productive_percentage = productivity_score / 100
        productive_time = actual_duration * productive_percentage
        wasted_time = actual_duration - productive_time

        # Detect repetition
        repetition_score = self.detect_repetition(transcript)
        if repetition_score > 30:
            wasted_reasons.append(f"High repetition detected ({repetition_score:.1f}% repeated content)")

        return {
            "summary": summary,
            "overall_sentiment": overall_sentiment,
            "sentiment_breakdown": {
                "positive": positive_count,
                "negative": negative_count,
                "neutral": neutral_count
            },
            "productivity_score": round(productivity_score, 1),
            "time_analysis": {
                "total_duration": f"{timedelta(seconds=int(actual_duration))}",
                "productive_time": f"{timedelta(seconds=int(productive_time))}",
                "wasted_time": f"{timedelta(seconds=int(wasted_time))}",
                "efficiency_percentage": round(productive_percentage * 100, 1)
            },
            "issues_detected": wasted_reasons,
            "transcript": transcript,
            "word_count": len(transcript.split()),
            "speaking_rate": f"{len(transcript.split()) / (actual_duration / 60):.1f} words/minute" if actual_duration > 0 else
        }


    def detect_repetition(self, text):
        """Detect repetitive content"""
        sentences = re.split(r'[.!?]+', text)
        sentences = [s.strip().lower() for s in sentences if len(s.strip()) > 10]

        if len(sentences) < 2:
            return 0

        repeated = 0
        for i, sent in enumerate(sentences):
            for j, other_sent in enumerate(sentences[i+1:], i+1):
                # Simple similarity check
                words1 = set(sent.split())
                words2 = set(other_sent.split())
                if words1 and words2:
                    similarity = len(words1.intersection(words2)) / len(words1.union(words2))
                    if similarity > 0.7:
                        repeated += 1
                        break

        return (repeated / len(sentences)) * 100

    def determine_overall_sentiment(self, positive, negative, neutral):
        """Determine overall sentiment with nuance"""
        total = positive + negative + neutral
        if total == 0:
            return "Neutral"
```

```
        pos_ratio = positive / total
        neg_ratio = negative / total

        if pos_ratio > 0.6:
            return "Very Positive"
        elif pos_ratio > 0.4:
            return "Positive"
        elif neg_ratio > 0.6:
            return "Very Negative"
        elif neg_ratio > 0.4:
            return "Negative"
        else:
            return "Neutral"

    def empty_results(self):
        """Return empty results structure"""
        return {
            "summary": "No content to analyze",
            "overall_sentiment": "Neutral",
            "sentiment_breakdown": {"positive": 0, "negative": 0, "neutral": 0},
            "productivity_score": 0,
            "time_analysis": {
                "total_duration": "0:00:00",
                "productive_time": "0:00:00",
                "wasted_time": "0:00:00",
                "efficiency_percentage": 0
            },
            "issues_detected": ["No audio content detected"],
            "transcript": "",
            "word_count": 0,
            "speaking_rate": "N/A"
        }

    def analyze_meeting(self, input_file=None, text_input=None):
        """Main entry point for analyzing a meeting"""
        try:
            if input_file:
                transcript_data = self.transcribe_audio(input_file)
            elif text_input:
                transcript_data = {"text": text_input, "duration": 0, "segments": []}
            else:
                raise ValueError("Provide either an input_file or text_input.")

            results = self.analyze_text(transcript_data)
            return results

        except Exception as e:
            print(f"❌ Error analyzing meeting: {e}")
            return self.empty_results()

    def generate_report(self, results):
        """Generate a formatted report"""
        report = f"""
=== 📋 MEETING ANALYSIS REPORT ===

📊 OVERVIEW:
• Duration: {results['time_analysis']['total_duration']}
• Words: {results['word_count']}
• Speaking Rate: {results['speaking_rate']}
• Productivity Score: {results['productivity_score']}%

📝 SUMMARY:
{results['summary']}


😊 SENTIMENT ANALYSIS:
• Overall: {results['overall_sentiment']}
• Positive: {results['sentiment_breakdown']['positive']}
• Negative: {results['sentiment_breakdown']['negative']}
• Neutral: {results['sentiment_breakdown']['neutral']}

⏰ TIME BREAKDOWN:
• Productive Time: {results['time_analysis']['productive_time']} ({results['time_analysis']['efficiency_percentage']}%)
• Wasted Time: {results['time_analysis']['wasted_time']}

⚠️ ISSUES DETECTED:
"""
```

```
        for issue in results['issues_detected'][:5]:
            report += f"• {issue}\n"

        if not results['issues_detected']:
            report += "• No major issues detected\n"

        return report

# Initialize the analyzer
print("🚀 Initializing Meeting Analyzer...")
analyzer = MeetingAnalyzer()

# File upload function
def upload_and_analyze():
    """Upload and analyze files"""
    print("\n📁 Click below to upload your audio/video file:")
    print("Supported formats: MP3, MP4, WAV, M4A, AVI, MOV, MKV, WEBM")

    uploaded = files.upload()

    if not uploaded:
        print("❌ No file uploaded!")
        return

    # Get the uploaded file name
    filename = list(uploaded.keys())[0]
    print(f"📁 Uploaded: {filename}")

    # Analyze the file
    print("\n🔍 Starting analysis...")
    results = analyzer.analyze_meeting(input_file=filename)

    # Display results
    print("\n" + "="*50)
    print(analyzer.generate_report(results))
    print("="*50)

    # Option to see full JSON
    show_json = input("\n📋 Show detailed JSON results? (y/n): ")
    if show_json.lower() in ['y', 'yes']:
        print("\n📊 DETAILED JSON RESULTS:")
        print(json.dumps(results, indent=2))

    # Clean up uploaded file
    if os.path.exists(filename):
        os.remove(filename)
        print(f"🗑️ Cleaned up {filename}")

# Alternative: Analyze text directly
def analyze_text_input():
    """Analyze text input directly"""
    print("\n📝 Enter your text to analyze:")
    text = input("Text: ")

    if not text.strip():
        print("❌ No text provided!")
        return

    print("\n🔍 Analyzing text...")
    results = analyzer.analyze_meeting(text_input=text)

    # Display results
    print("\n" + "="*50)
    print(analyzer.generate_report(results))
    print("="*50)

print("\n✅ Ready! Choose an option:")
print("1. Upload audio/video file")
print("2. Analyze text directly")

choice = input("\nEnter choice (1 or 2): ")

if choice == "1":
    upload_and_analyze()
elif choice == "2":
    analyze_text_input()
```

```
    else:
        print("❌ Invalid choice. Run the cell again and choose 1 or 2.")
```

```
🚀 Initializing Meeting Analyzer...
🔄 Loading AI models...
100%|██████████████████████████████████████| 139M/139M [00:01<00:00, 124MiB/s]
config.json:        1.58k/? [00:00<00:00, 28.7kB/s]
model.safetensors: 100%                                  1.63G/1.63G [00:49<00:00, 25.7MB/s]
generation_config.json: 100%                                363/363 [00:00<00:00, 43.8kB/s]
vocab.json:        899k/? [00:00<00:00, 26.9MB/s]
merges.txt:        456k/? [00:00<00:00, 29.5MB/s]
tokenizer.json:        1.36M/? [00:00<00:00, 47.4MB/s]
Device set to use cuda:0
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f (https://h
Using a pipeline without specifying a model name and revision in production is not recommended.
config.json: 100%                                        629/629 [00:00<00:00, 40.1kB/s]
model.safetensors: 100%                                  268M/268M [00:06<00:00, 25.5MB/s]
tokenizer_config.json: 100%                                48.0/48.0 [00:00<00:00, 5.33kB/s]
```

----Drafts from here----

```
#Draft 1 of meetIQ from here
!pip install openai-whisper transformers moviepy torch
```

```
import torch
print("GPU Available:", torch.cuda.is_available())
```

```
GPU Available: True
Enter choice (1 or 2): 1
```

```
import whisper
import moviepy.editor as mp
from transformers import pipeline
from datetime import timedelta
import json
from google.colab import files

device = 0 if torch.cuda.is_available() else -1

# -------------------------------
# STEP 1: TRANSCRIPTION
# -------------------------------
def transcribe_audio(file_path):
    """Convert mp3/mp4 to text using Whisper"""
    if file_path.endswith(".mp4"):
        clip = mp.VideoFileClip(file_path)
        audio_path = "temp_audio.mp3"
        clip.audio.write_audiofile(audio_path)
        file_path = audio_path

    model = whisper.load_model("base", device="cuda" if torch.cuda.is_available() else "cpu")
    result = model.transcribe(file_path)
    return result["text"]

# -------------------------------
# STEP 2: SENTIMENT & SUMMARY
# -------------------------------
def analyze_text(transcript):
    """Perform summarization, sentiment analysis, and productivity scoring"""

    summarizer = pipeline("summarization", model="facebook/bart-large-cnn", device=device)
    summary = summarizer(transcript, max_length=150, min_length=50, do_sample=False)[0]['summary_text']

    sentiment_analyzer = pipeline("sentiment-analysis", device=device)

    # Break transcript into chunks (simulate conversation turns)
    sentences = transcript.split(". ")
    wasted_reasons = []
    positive, negative, neutral = 0, 0, 0

    for s in sentences:
        if not s.strip():
            continue
        sentiment = sentiment_analyzer(s[:512])[0]  # truncate long text for model
        label = sentiment['label']
```